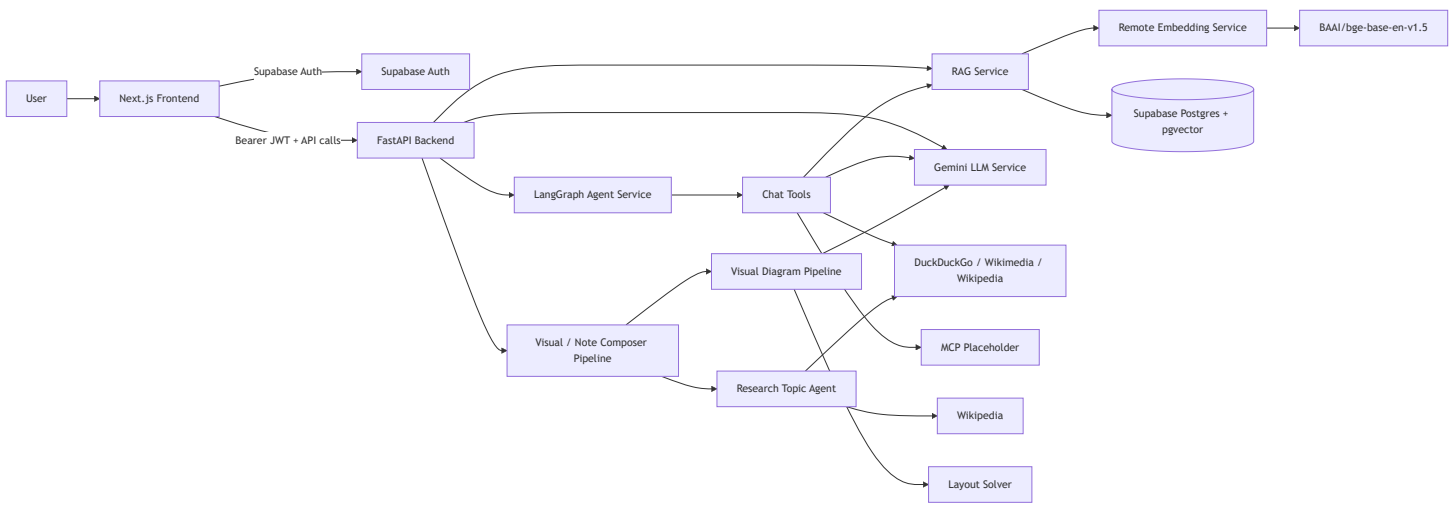


# CognifyAI Architecture

CognifyAI is a RAG-based adaptive learning platform. It combines a Next.js frontend, a FastAPI backend, Supabase authentication, Supabase/PostgreSQL with pgvector, a remote embedding service, Gemini-based LLM workflows, and a LangGraph tutor agent.

## High-Level Architecture



## Main Components

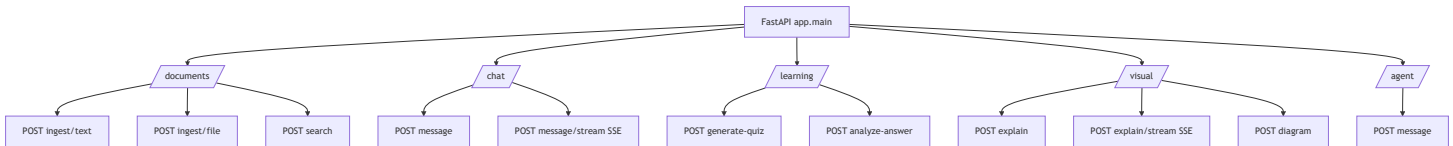
| Layer               | Technology                                 | Role                                                                                      |
|---------------------|--------------------------------------------|-------------------------------------------------------------------------------------------|
| Frontend            | Next.js 16, React 19, TypeScript, Tailwind | Dashboard UI, chat, uploads, quiz, visual explanations                                    |
| Auth                | Supabase Auth                              | Login/session handling; frontend sends access token to backend                            |
| Backend API         | FastAPI                                    | REST + SSE API for documents, chat, learning, visual explanations                         |
| Agent Orchestration | LangGraph                                  | Routes chat requests into study, quiz, analyze, visual, web/image search, MCP placeholder |
| LLM                 | Gemini API                                 | Routing, chat, quiz generation, misconception analysis, note writing, diagram planning    |

| Layer           | Technology                               | Role                                                          |
|-----------------|------------------------------------------|---------------------------------------------------------------|
| RAG             | Supabase + pgvector                      | Stores document chunks and performs semantic search           |
| Embeddings      | Hugging Face Space microservice          | Remote BAAI/bge-base-en-v1.5 embeddings                       |
| External Search | DuckDuckGo, Wikimedia Commons, Wikipedia | Web/image/research enrichment                                 |
| MCP             | Placeholder backend node                 | Intent exists, but no real MCP connector adapter is wired yet |

## Important Files

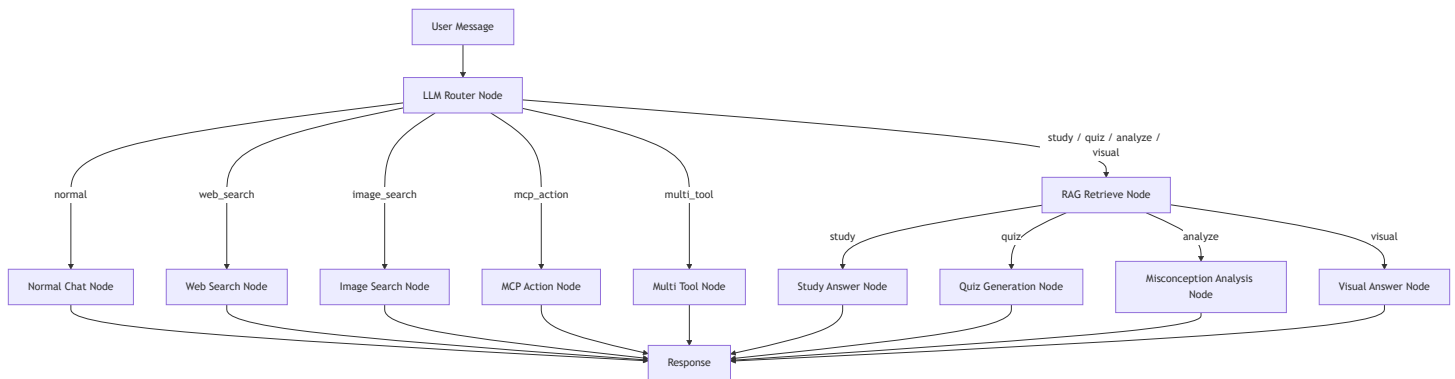
| Area                     | File                                      |
|--------------------------|-------------------------------------------|
| Backend entry            | backend/app/main.py                       |
| Agent graph              | backend/app/services/agent_service.py     |
| RAG service              | backend/app/services/rag_service.py       |
| Embedding client         | backend/app/services/embedding_service.py |
| LLM service              | backend/app/services/llm_service.py       |
| Database schema          | backend/supabase_migration.sql            |
| Frontend API client      | frontend/src/lib/api.ts                   |
| Frontend Supabase client | frontend/src/lib/supabase/client.ts       |
| Embedding microservice   | embedding-space/cognify-embedding/app.py  |

## Backend API Surface



# Agent Workflow

The backend agent is implemented in `backend/app/services/agent_service.py`. It uses LangGraph when available, and has a fallback imperative runner if LangGraph is not installed.



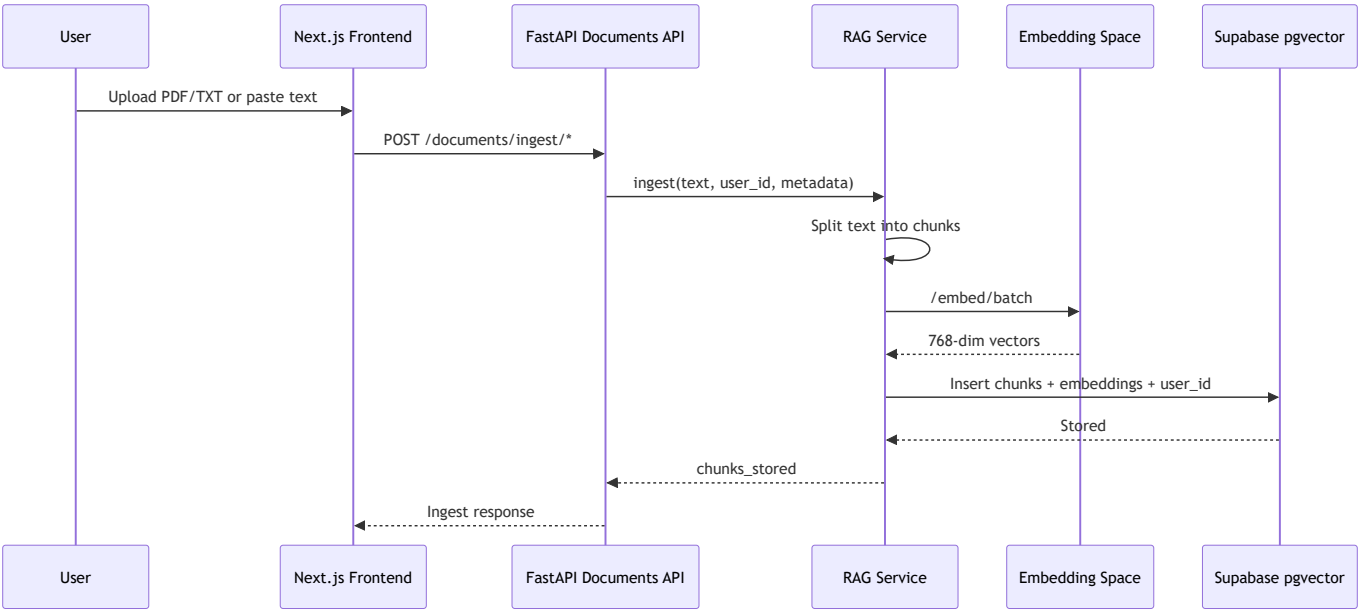
Agent intents:

- normal
- study
- quiz
- analyze
- visual
- web\_search
- image\_search
- mcp\_action
- multi\_tool

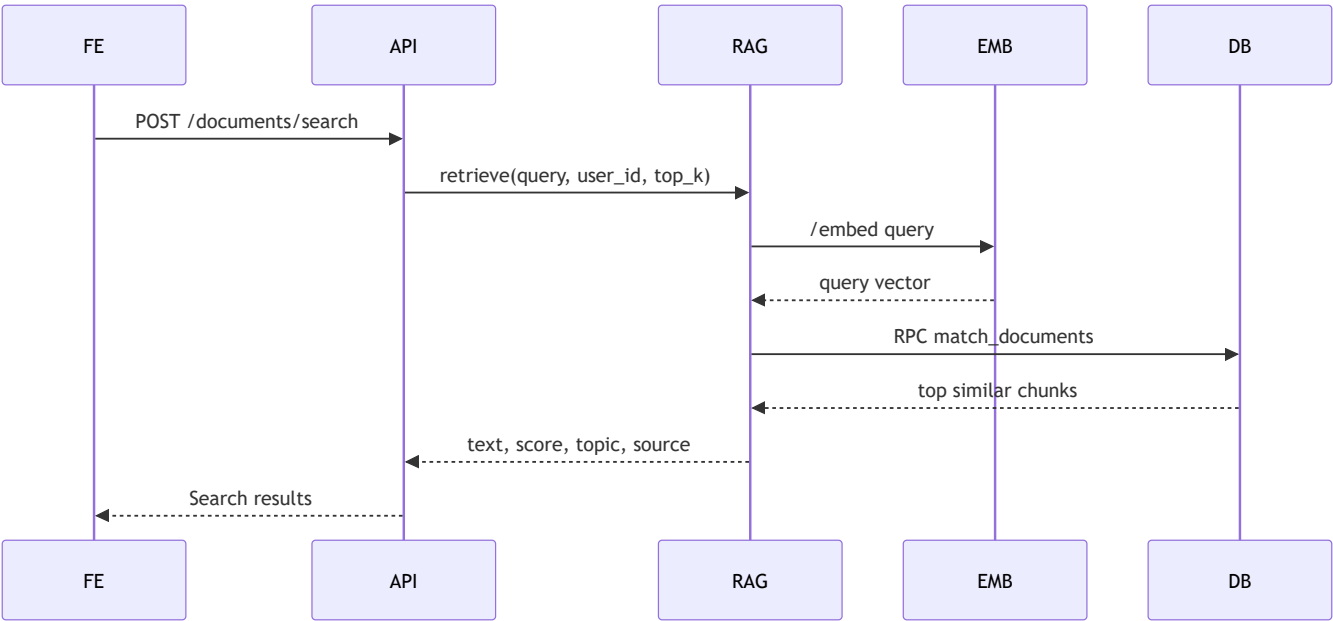
The `mcp_action` route exists, but `backend/app/agents/chat_tools/mcp.py` currently returns a setup-pending response. There is no live MCP connector adapter wired into the FastAPI backend yet.

# RAG / Document Workflow

## Ingestion



## Semantic Search



## Database Model

The Supabase migration is defined in `backend/supabase_migration.sql`.

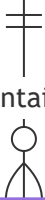
| documents   |            |    |
|-------------|------------|----|
| uuid        | id         | PK |
| uuid        | user_id    |    |
| text        | text       |    |
| varchar     | topic      |    |
| varchar     | source     |    |
| vector      | embedding  |    |
| timestamptz | created_at |    |

| users       |              |    |
|-------------|--------------|----|
| uuid        | id           | PK |
| varchar     | name         |    |
| varchar     | learner_type |    |
| timestamptz | created_at   |    |



has

| quiz_sessions |              |    |
|---------------|--------------|----|
| uuid          | id           | PK |
| uuid          | user_id      | FK |
| varchar       | topic        |    |
| varchar       | learner_type |    |
| timestamptz   | created_at   |    |



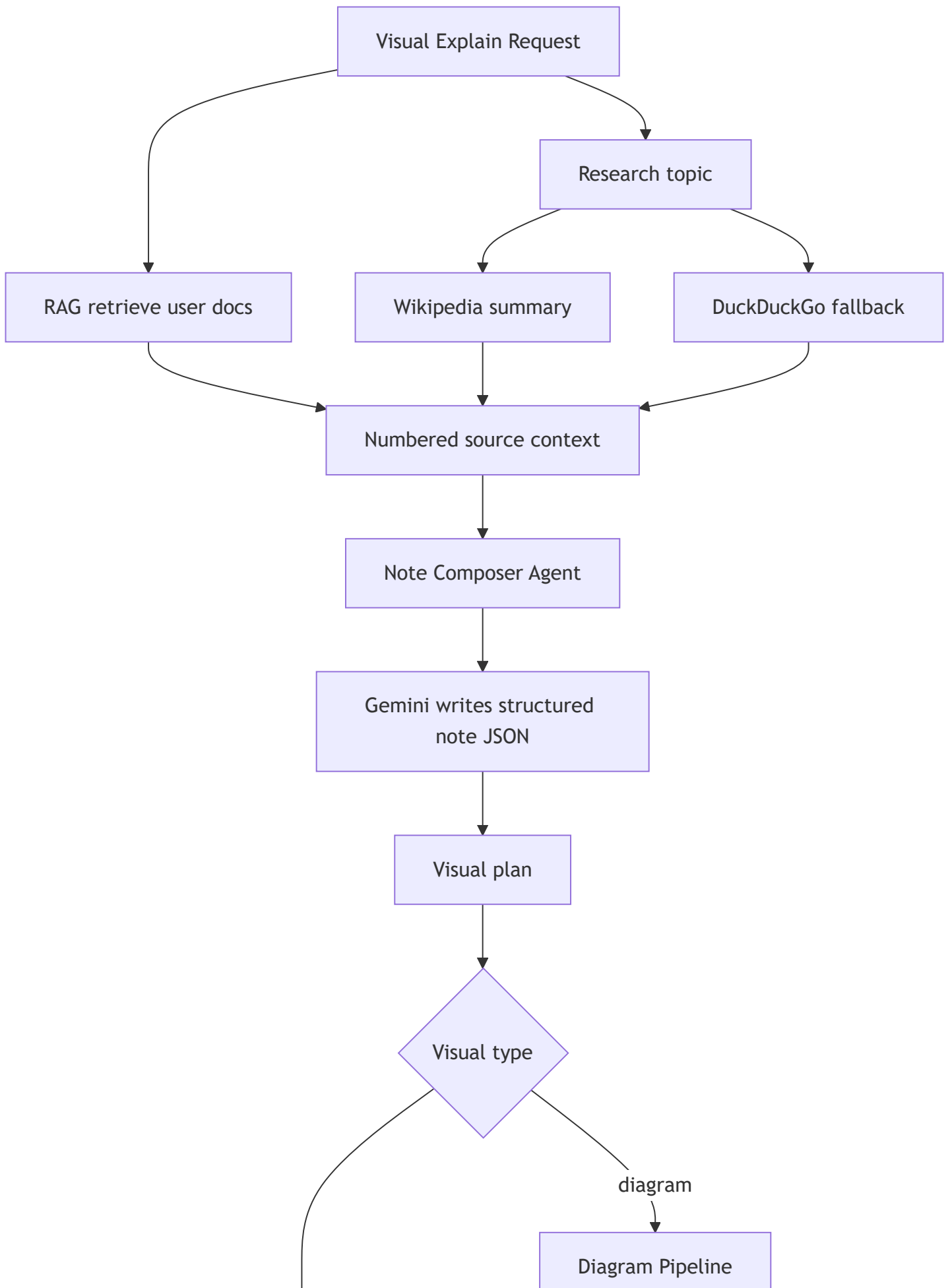
contains

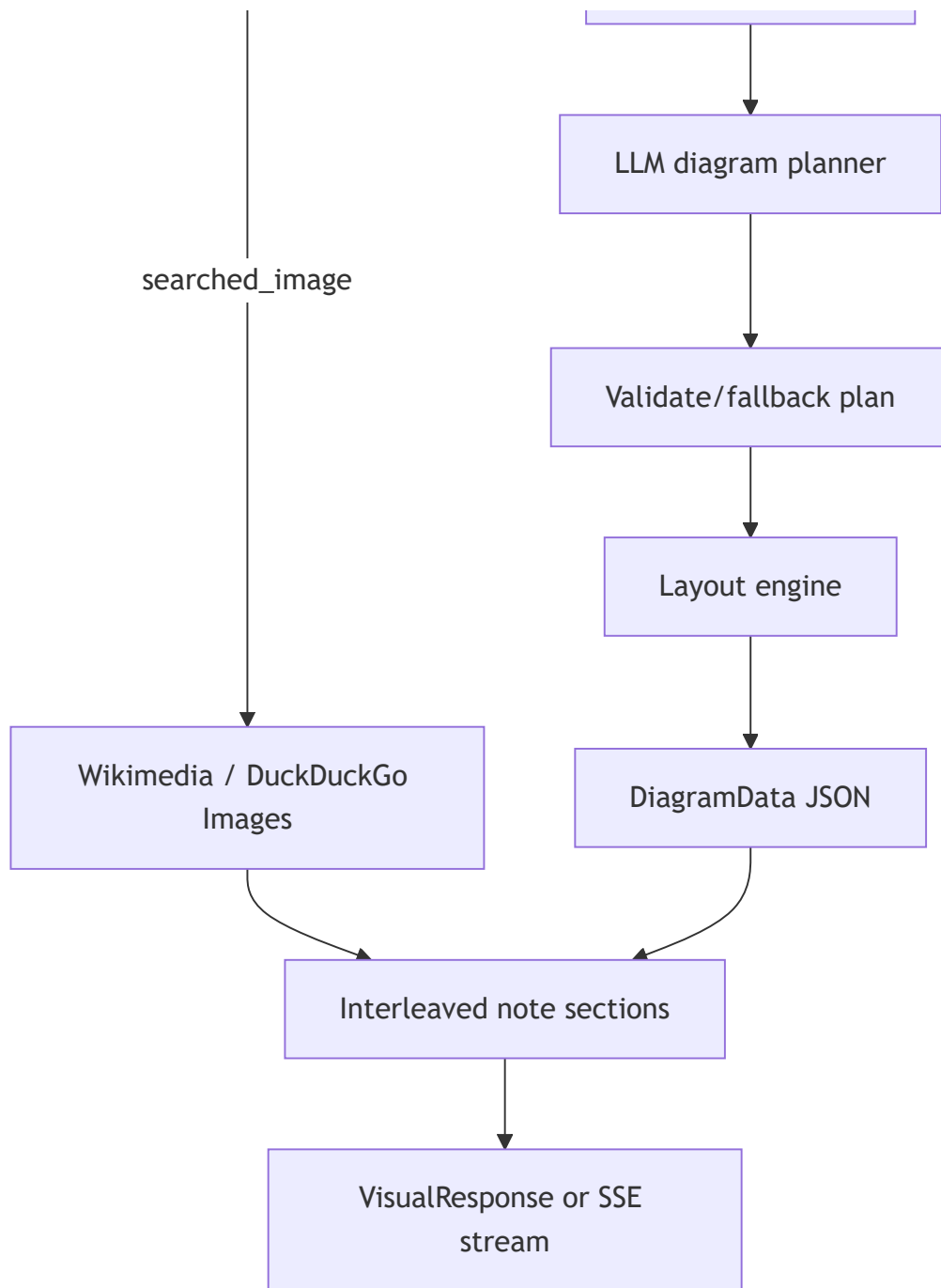
| quiz_attempts |            |    |
|---------------|------------|----|
| uuid          | id         | PK |
| uuid          | session_id | FK |
| text          | question   |    |

|            |                   |  |
|------------|-------------------|--|
| text       | selected_answer   |  |
| text       | correct_answer    |  |
| boolean    | is_correct        |  |
| varchar    | misconception_tag |  |
| timestampz | created_at        |  |

The key database function is `match_documents(...)` , which performs cosine similarity search using `pgvector` and filters by `user_id` .

# Visual Explanation Workflow





## Embedding Microservice

The embedding service is a separate FastAPI app under `embedding-space/cognify-embedding/app.py`.

It loads:

`BAAI/bge-base-en-v1.5`

Endpoints:

- `GET /health`

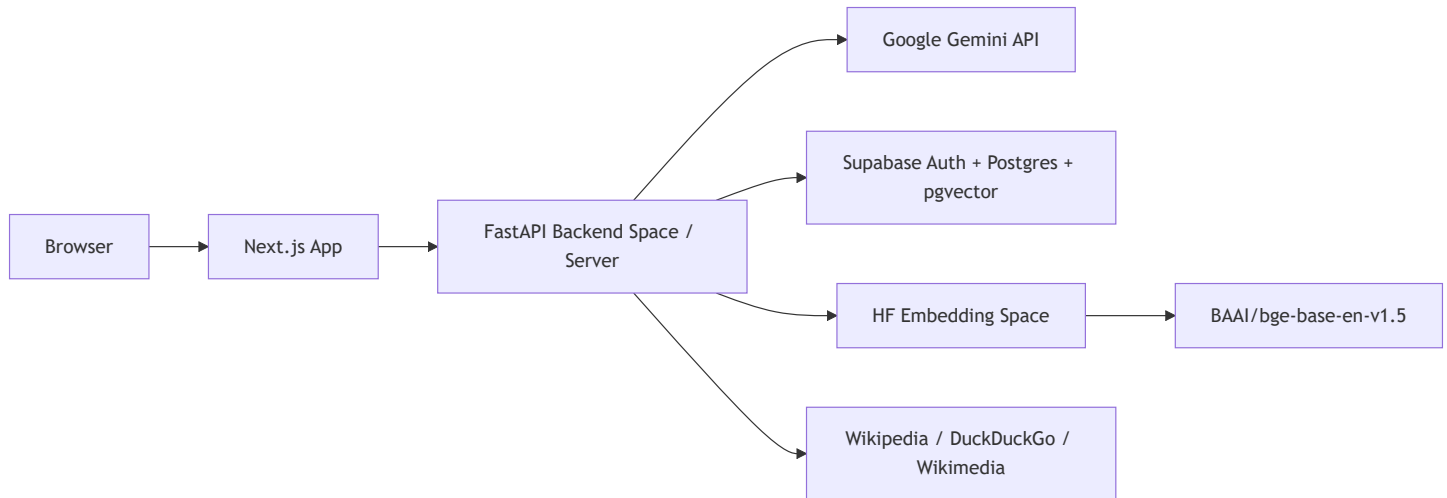
- `POST /embed`
- `POST /embed/batch`

The main backend does not load `torch` , `transformers` , or `sentence-transformers` directly. It calls the remote embedding service through `EMBEDDING_SERVICE_URL` .

## Agent Tools

| Tool           | File                                                       | What it does                                    |
|----------------|------------------------------------------------------------|-------------------------------------------------|
| Router         | <code>backend/app/agents/chat_tools/router.py</code>       | Uses Gemini to classify user intent             |
| Retrieval      | <code>backend/app/agents/chat_tools/retrieval.py</code>    | Pulls user-scoped RAG context                   |
| Normal chat    | <code>backend/app/agents/chat_tools/normal.py</code>       | General conversation                            |
| Tutoring/study | <code>backend/app/agents/chat_tools/tutoring.py</code>     | RAG-grounded explanation                        |
| Quiz           | <code>backend/app/agents/chat_tools/quiz.py</code>         | Generates MCQs                                  |
| Analysis       | <code>backend/app/agents/chat_tools/analysis.py</code>     | Misconception feedback                          |
| Visual         | <code>backend/app/agents/chat_tools/visual.py</code>       | Diagram/visual response                         |
| Web search     | <code>backend/app/agents/chat_tools/web_search.py</code>   | DuckDuckGo text search                          |
| Image search   | <code>backend/app/agents/chat_tools/image_search.py</code> | DuckDuckGo + Wikimedia images                   |
| MCP            | <code>backend/app/agents/chat_tools/mcp.py</code>          | Placeholder only; no live connector adapter yet |

# Deployment Shape



## Summary

CognifyAI is structured as a full-stack AI learning platform. The frontend authenticates users with Supabase and sends bearer-token API requests to FastAPI. The backend validates Supabase JWTs, retrieves or stores user-scoped learning documents, calls a remote embedding service for vector representations, searches Supabase pgvector data, and uses Gemini for chat, routing, quiz generation, misconception analysis, visual-note composition, and diagram planning.

The strongest implemented workflows are:

- Document ingestion and semantic search
- RAG-grounded chat
- Adaptive quiz generation
- Misconception analysis
- Visual note generation with diagrams/images
- Web and image search enrichment

MCP support is represented architecturally, but it is currently a placeholder and needs a backend connector adapter before it can call real private tools.